

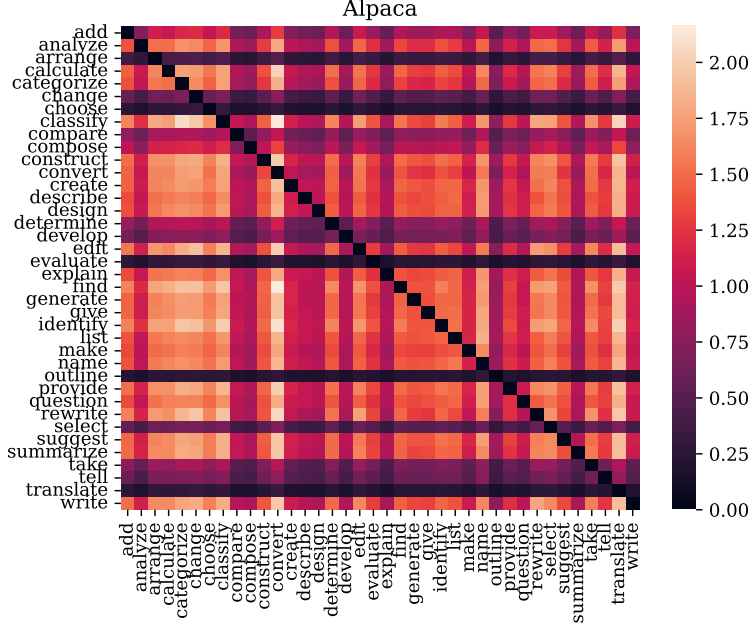


Figure 8: Alpaca heatmap where i, j th entry is $\max(0, \delta_j^i)$ (the change in loss on s_j after training on s_i for 150 steps). Diagonal entries are set to 0 for clearer visualization.

C.2 Graph Learning Details

We describe how the skills graph was learned on each dataset.

- Alpaca (Figure 8): we use Algorithm 3 and train for $K = 150$ steps per skill. Each edge $i \rightarrow j$ has a weight of δ_j^i , the difference in loss on skill j before and after training on i . Next, we compare the average validation loss of skill-stratified sampling versus random sampling when we train for $K = 1000$ steps. We find that skill-stratified sampling only does 0.007 better than random sampling, confirming that Alpaca’s dense skills graph suggests that random sampling is the best we can do.
- Pile of Law (Figure 9): we use Algorithm 3 and train for $K = 150$ steps. Each edge $i \rightarrow j$ has a weight of δ_j^i , the difference in loss on skill j before and after training on i .
- LEGO (Figure 10): we use both Algorithm 2 and Algorithm 3 and train for $K = 6000$ steps each. Each edge $i \rightarrow j$ has a weight of 0.5 if the amount of data associated with skill j that is needed to reach 0.01 validation loss is less when training on (i, j) than on j (edges are set to 0 if 0.01 validation loss is not reached, even if loss is decreasing). Each edge $i \rightarrow j$ is also set to 0.5 if training on i decreases loss directly on j . We set each diagonal entry of A to be 1.
- Addition (Figure 11): we use Algorithm 2 and train for $K = 6000$ steps. Each edge $i \rightarrow j$ has a weight of 0.5 if the amount of data associated with skill j that is needed to reach 0.01 validation loss is less when training on (i, j) than on j (edges are set to 0 if 0.01 validation loss is not reached, even if loss is decreasing). We set each diagonal entry of A to be 1.
- Natural Instructions (Figure 12, 13, 14): we use Algorithm 3. For the pre-training setting, we train for $K = 600$ steps and assign each edge $i \rightarrow j$ a weight δ_j^i equal to the change in loss on j in the first 100 steps for all $i, j \in [k]$, including diagonal entries. For the fine-tuning setting, we train for $K = 600$ steps and assign each edge $i \rightarrow j$ a weight δ_j^i equal to the change in loss before and after training. For the out-of-domain setting, we train for $K = 600$ steps and assign each edge $i \rightarrow j$ a weight δ_j^i equal to the change in loss before and after training in the first 100 steps.
- RedPajama (Figure 15): we use Algorithm 3 and train for 1 billion tokens per data source. We assign each edge $i \rightarrow j$ a weight δ_j^i equal to the change in perplexity on the validation data before and after training.

C.3 Training Details

We describe the parameters used for SKILL-IT.

SKILL-IT pre-training

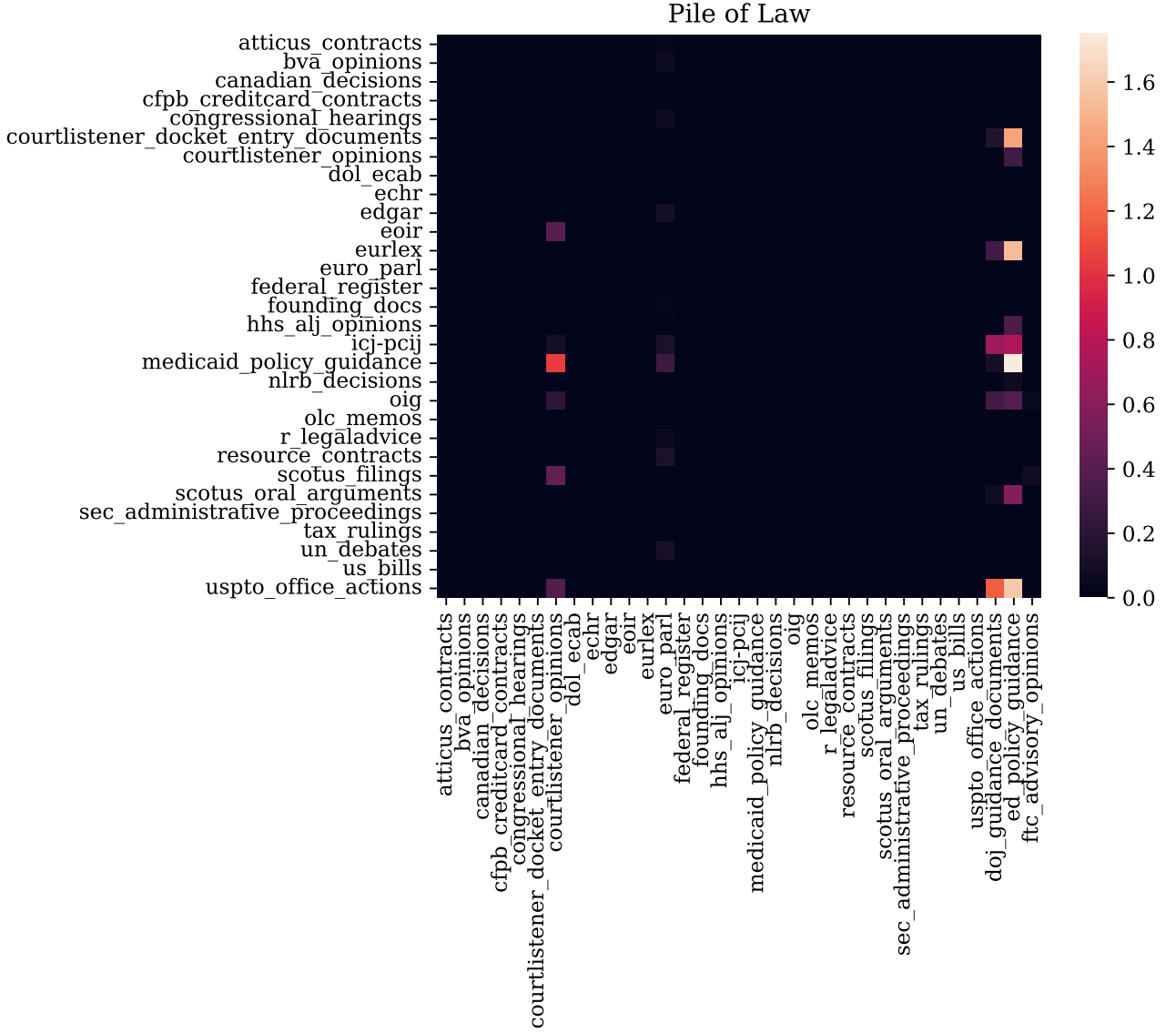


Figure 9: Pile of Law heatmap where i, j th entry is $\max(0, \delta_j^i)$ (the change in loss on s_j after training on s_i for 150 steps). Diagonal entries are set to 0 for clearer visualization.

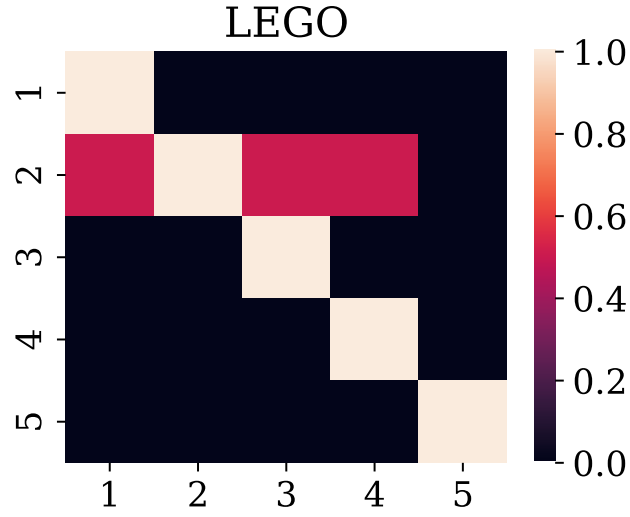


Figure 10: LEGO heatmap with $k = 5$ where i, j th entry is set to 0.5 if the number of steps needed to reach 0.01 loss on skill j when training on a balanced mixture of skills i and j is less than when training on skill j only.

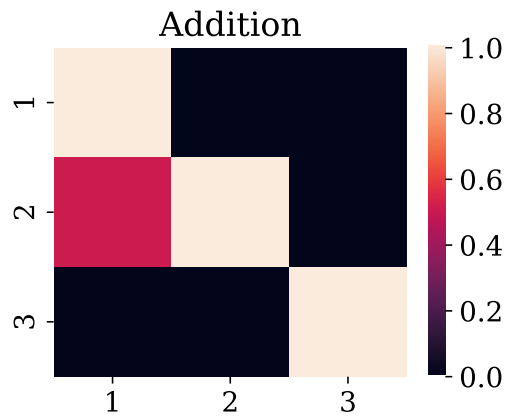


Figure 11: Addition heatmap with $k = 3$ where i, j th entry is set to 0.5 if the number of steps needed to reach 0.01 loss on skill j when training on a balanced mixture of skills i and j is less than when training on skill j only.